

Chapter 12: Objects

By Victoria Gonda

Kotlin introduces a new keyword that is unavailable in other languages Kotlin is often compared to, such as Java and Swift: The `object` keyword.

Like Java and Swift, Kotlin supports **object-oriented programming**. As you saw in Chapter 11, “Classes,” you use classes to define custom types, and instances of classes are called **objects**.

Kotlin uses `object` to denote a custom type that can only have a **single instance**. The name choice for the `object` keyword can sometimes lead to confusion with class instances, since they’re also called objects. As you’ll see in this chapter, you can also use `object` to create **anonymous objects**, for which a new instance is created each time the anonymous object is used, another potential source of confusion.

This chapter will help you make sense of this potential confusion.

In discussing your code, you’ll often have to rely on the context to determine whether an “object” is a class instance, the single instance of an entity created using `object` or an anonymous object.

The `object` keyword lets you easily implement a common pattern in software development: The **singleton** pattern.

Singletons

The **singleton** pattern is one of the more straightforward of the software engineering **design patterns**. In most object-oriented languages, the pattern is used when you want to restrict a class to have a single instance during any given run of an application.

There are a few hurdles you must typically jump in order to create a singleton, in terms of setting up the single instance and performing the restriction to one object. Use of singletons is sometimes discouraged because they introduce a **global state** into your application; therefore, you must be careful when accessing a singleton from different application threads. But singletons are useful in certain use cases wherein the scope of use is limited.

Kotlin addresses some of these concerns by giving you a built-in way to create singletons.

Named objects

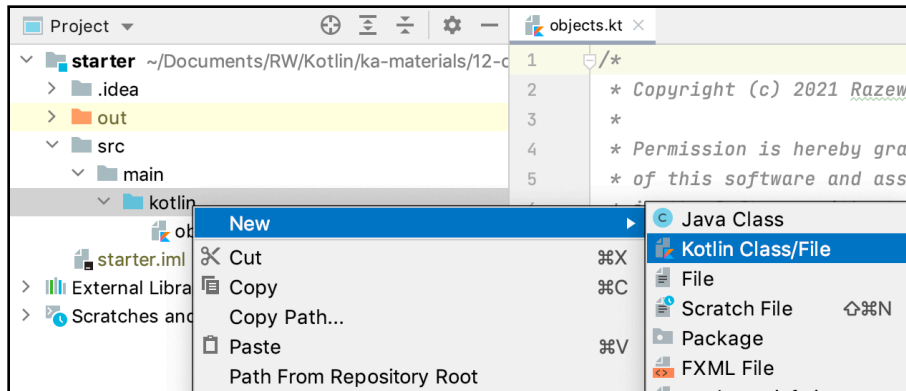
The `object` keyword in Kotlin lets you define a type that only has a single instance — a **named object**.

A type defined with `object` cannot have constructors: Since there is only one instance of an object, there is no reason to provide constructor functions to create other instances. In a sense, the type *is* the instance.

To see what the Kotlin compiler is doing to follow the singleton pattern, it's instructive to see how the compiled version of Kotlin code created with `object` looks in Java. You can do so by having IntelliJ IDEA **decompile** the Kotlin bytecode into Java.

Getting started

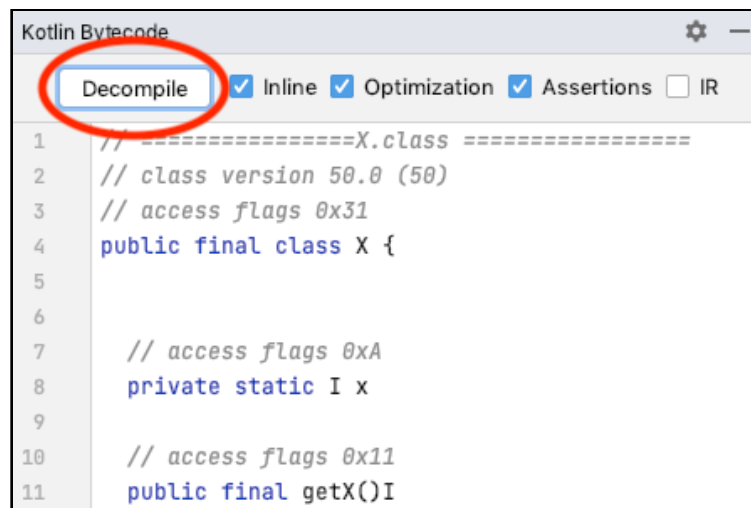
Open the chapter starter project and create a new file by right-clicking on the **src** folder and choosing **New ▶ Kotlin File/Class** and naming the file **X**:



Add the following bare bones object into the new file:

```
object X {  
    var x = 0  
}
```

Choose **Tools ▶ Kotlin ▶ Show Kotlin Bytecode**, which will open the **Kotlin Bytecode** panel:



Next, hit the **Decompile** button in the panel. The decompiled Java for the simple object will open in a new editor window:

```
public final class X {  
    private static int x;  
    public static final X INSTANCE;  
  
    public final int getX() {  
        return x;  
    }  
  
    public final void setX(int var1) {  
        x = var1;  
    }  
  
    static {  
        X var0 = new X();  
        INSTANCE = var0;  
    }  
}
```

The Java code uses a common approach to create singletons in Java. You have the static and final INSTANCE field that is of the same type as the class.

The INSTANCE value is set in a static block, and it is set to a new instance of the class. You also have getters and setters for the single-member field x.

Comparing the Java and Kotlin versions of the code, you see that the boilerplate singleton setup code has been significantly reduced by using the object keyword.

Singleton use cases

An example use case for a singleton is an in-memory repository for a set of data. Consider an app that needs a registry of students who are defined with the following data class. Add this class to **objects.kt**:

```
data class Student(  
    val id: Int,  
    val firstName:  
    String, val  
    lastName: String  
) {  
    {  
        var fullName = "$lastName, $firstName"  
    }  
}
```

And this to the `main()` function:

```
val marie = Student(1, "Marie", "Curie")
val albert = Student(2, "Albert", "Einstein")
val emmy = Student(3, "Emmy", "Noether")
```

This creates your brilliant students, but you still need your registry.

Add the following `StudentRegistry` outside of the `main()` function:

```
object StudentRegistry {
    val allStudents = mutableListOf<Student>()

    fun addStudent(student: Student) {
        allStudents.add(student)
    }

    fun removeStudent(student: Student) {
        allStudents.remove(student)
    }

    fun listAllStudents() {
        allStudents.forEach {
            println(it.fullName)
        }
    }
}
```

Using object, you create a registry that:

1. Maintains the list of students in a mutable list.
2. Lets you add and remove students from the registry.
3. Lets you print out the full name of all the students in the registry.

Now, add the following to the bottom of your `main()` function:

```
StudentRegistry.addStudent(marie)
StudentRegistry.addStudent(albert)
StudentRegistry.addStudent(emmy)

StudentRegistry.listAllStudents()
// > Curie, Marie
// > Einstein, Albert
// > Noether, Emmy
```

This adds your students to the registry. You call methods defined on the object using the object name with dot syntax.

Run the code to see your students printed out.

Had you used a class to represent your student registry, your app would allow for multiple registries to be created, which could lead to inconsistent registries to exist within the software. Using a Kotlin object ensures that only one registry can be created.

Another example use case is to use object to provide a **namespace** for constants and methods that need to be referenced from multiple places in your app:

```
object JsonKeys {  
    const val JSON_KEY_ID = "id"  
    const val JSON_KEY_FIRSTNAME = "first_name"  
    const val JSON_KEY_LASTNAME = "last_name"  
}
```

This creates a namespace for holding JSON keys that will be used to parse JSON received from a server. By putting constants into an object, you reduce the likelihood of name collisions when your constants are given commonly used names.

Comparison to classes

While constructors are not allowed for objects, they do have many similarities with classes:

- Objects can have properties and member functions.
- Properties of the object must be initialized before use, either at declaration or in an `init` block.
- Objects can inherit from classes and implement interfaces.

Using static members

One of the students you define in this chapter, **Emmy Noether**, was a key contributor to the theory of conservation laws in physics. There appears to be a “law of conservation of keywords” because, while Kotlin has gained the `object` keyword, it’s also lost a keyword found in other languages like Java and Swift: There is no `static` keyword in Kotlin.

*Portrait of Emmy Noether*

The `static` keyword is used in these other languages to denote a class member that is common to all instances of the class and is not specific to each instance. Static members remove the need to duplicate items that are common to all instances.

But removing code duplication is useful, so how does Kotlin allow you to define static members? You do so by creating a **companion object** inside the class.

Creating companion objects

You create the companion object by prepending `companion` to an object defined in the class. Add this class to your file:

```
class Scientist private constructor(  
    val id: Int,  
    val firstName: String,  
    val lastName: String  
) {  
  
    companion object {  
        var currentId = 0  
  
        fun newScientist(  
            firstName: String,  
            lastName: String  
        ): Scientist {  
            currentId += 1  
            return Scientist(currentId, firstName, lastName)  
        }  
    }  
  
    var fullName = "$firstName $lastName"  
}
```

In the `Scientist` class, you've included a companion object that holds a `currentId` value that you'll use for generating unique ID numbers for each scientist. The `currentId` value is common to all instances of the class, and it is used by the class to create new ID values when a new scientist instance is created.

A common use case for static members is to implement the **factory** pattern for creating new class instances. You're using the factory pattern in `Scientist` by making the class primary constructor private and adding a factory method `newScientist()` to the companion object, which creates new scientist instances.

By making the constructor private, you enforce that the new scientist instances can only be created using the factory method, ensuring that your `currentId` value is correctly incremented whenever new scientist objects are instantiated.

You can create a repository of scientists as a singleton. Add the following `ScientistRepository`:

```
object ScientistRepository {  
    val allScientists = mutableListOf<Scientist>()  
  
    fun addScientist(scientist: Scientist) {  
        allScientists.add(scientist)  
    }  
  
    fun removeScientist(scientist: Scientist) {  
        allScientists.remove(scientist)  
    }  
  
    fun listAllScientists() {  
        allScientists.forEach {  
            println("${it.id}: ${it.fullName}")  
        }  
    }  
}
```

This repository is similar to your `StudentRegistry`, with the exception of the format used to print the names.

Then, create your scientists at the bottom of your `main()` function:

```
val emmy = Scientist.newScientist("Emmy", "Noether")  
val isaac = Scientist.newScientist("Isaac", "Newton")  
val nick = Scientist.newScientist("Nikola", "Tesla")  
  
ScientistRepository.addScientist(emmy)  
ScientistRepository.addScientist(isaac)  
ScientistRepository.addScientist(nick)  
  
ScientistRepository.listAllScientists()
```



```
// 1: Emmy Noether  
// 2: Isaac Newton  
// 3: Nikola Tesla
```

You create new scientist instances using dot syntax to call the companion object method on the class name. Run your additions to see the names printed out with correctly incremented IDs.

Companion naming and accessing from Java

The companion object is given an implicit name of `Companion`. You can use a custom name by adding it after the `companion object` keywords:

```
companion object Factory {  
    // companion object members  
}
```

You'll see in Chapter 14, "Methods," how the companion object name is used to extend the capabilities of the companion object.

Using the companion object name when accessing companion object members is redundant in Kotlin code. When calling the Kotlin companion object code from Java, however, you must use the companion object name:

```
// java  
Scientist isaac =  
    Scientist.Factory.newScientist("Isaac", "Newton");
```

If the companion object has not been given a custom name, you'll use the implicit name `Companion` instead.

Mini-exercise

Update the `Student` data class from above to keep track of how many students have been created. Use a companion object method `numberOfStudents()` to get the number of student instances.

Hint: use the `init` block to increment a counter.

If you get stuck, you can look at the challenge project for this chapter.

Using anonymous objects

Anonymous classes are used in Java to override the behavior of existing classes without the need to subclass, and also to implement interfaces without defining a concrete class. In both cases, the compiler creates a single anonymous instance, to which no name need be given. You'll learn more about inheritance in Chapter 15, "Advanced Classes," and interfaces in Chapter 17, "Interfaces."

You use `object` to create the Kotlin version of anonymous classes called **anonymous objects** or **object expressions**.

Suppose you had an interface that let you keep track of how many students and scientists you have in your app. Add this to your file:

```
interface Counts {  
    fun studentCount(): Int  
    fun scientistCount(): Int  
}
```

This interface has two method signatures that an instance needs to implement.

Create an anonymous object using this interface at the bottom of `main()`:

```
val counter = object : Counts {  
    override fun studentCount(): Int {  
        return StudentRegistry.allStudents.size  
    }  
  
    override fun scientistCount(): Int {  
        return ScientistRepository.allScientists.size  
    }  
}  
  
println(counter.studentCount()) // > 3  
println(counter.scientistCount()) // > 3
```

You create an instance of the counter using the `object` keyword followed by a colon and the name of the interface. Inside braces, you override each of the interface methods. Run this code so you can see the counts print out.

Unlike named objects, which act as singletons, there will be a different version of an anonymous object in your app each time one is created.

If you were to follow the decompile steps from above to see the Java version of the Kotlin code, you end up with the following Java code for the anonymous object:

```
<undefinedtype> counter = new Counts() {  
    public int studentCount() {  
        return StudentRegistry.INSTANCE.getAllStudents().size();  
    }  
  
    public int scientistCount() {  
        return ScientistRepository.INSTANCE  
            .getAllScientists().size();  
    }  
};
```

So the Kotlin compiler is just creating an anonymous Java class for the anonymous object.

Challenges

1. Create a named object that lets you check whether a given `Int` value is above a threshold. Name the object `Threshold` and add a method `isAboveThreshold(value: Int)`.
2. Create a version of the `Student` class that uses a factory method `loadStudent(studentMap: Map<String, String>)` to create a student with a first and last name from a map such as `mapOf("first_name" to "Neils", "last_name" to "Bohr")`. Default to using “First” and “Last” as the names if the map not contain a first name or last name.
3. Create an anonymous object that implements the following interface:

```
interface ThresholdChecker {  
    val lower: Int  
    val upper: Int  
  
    /**  
     * Returns true if value is higher than the upper threshold  
     * and false otherwise  
     */  
    fun isLit(value: Int): Boolean  
    /**  
     * Returns true if value is less than the lower threshold  
     * and false otherwise  
     */  
    fun tooQuiet(value: Int): Boolean  
}
```

Use a lower value of 7 and an upper value of 10 in the anonymous object.

Key points

- The **singleton pattern** is used when you want only one instance of a type to be created in your app.
- The `object` keyword is unique to Kotlin compared with similar languages, and it gives you a built-in way to make **singletons** with **named objects**. It also lets you make **anonymous objects**, the Kotlin version of Java anonymous classes.
- A class **companion object** gives you the Kotlin equivalent of Java static members.
- **Anonymous objects** — or **object expressions** — let you create unnamed instances of interfaces and to override class behavior without subclassing.
- Using **Show Kotlin Bytecode** and decompiling in IntelliJ IDEA is an informative way to understand what the Kotlin compiler is doing.

Where to go from here?

As you’ve seen in this chapter, just like classes, objects have properties and methods, and there’s more to learn about for both. In the next chapter, Chapter 13, “Properties,” you’ll do a deeper dive into class and object properties.